

Traditional-DIC

Operation Manual

English edition

Generated from the project documentation

Contents

Traditional-DIC User Manual

Contents

Project Architecture

Supported Workflows

Installation and Build

Configuration and Case Layout

Running Complete Cases

Beginner Checklist

Configuration Walkthrough

Reproducible Case Procedure

Results and Strain

Python API Reference

Troubleshooting

Traditional-DIC User Manual

Contents

- [Project architecture](#project-architecture)
- [Supported workflows](#supported-workflows)
- [Installation and build](#installation-and-build)
- [Configuration and case layout](#configuration-and-case-layout)
- [Running complete cases](#running-complete-cases)
- [Results and strain](#results-and-strain)
- [Python API reference](#python-api-reference)
- [Beginner checklist](#beginner-checklist)
- [Configuration walkthrough](#configuration-walkthrough)
- [Reproducible case procedure](#reproducible-case-procedure)
- [Troubleshooting](#troubleshooting)

Project Architecture

Traditional-DIC has a C++17 numerical core and a pybind11 extension consumed by the Python applications. The data flow is:

```
images + ROI -> preprocessing/B-spline -> initialization
               -> Subset-DIC or Mesh-DIC -> 2D fields
               -> calibration/geometry -> stereo or multi-view reconstruction
               -> 3D displacement, strain, CSV/JSON, visualization
```

include/dic/ contains public C++ interfaces and src/ their implementation. bindings/python/ compiles `_traditional_dic`; python/traditional_dic/ is the public Python workflow layer. examples/ are complete applications, not merely demonstrations.

The shared layer owns grayscale images, binary masks, B-spline interpolation, SSD/ZNSSD/ZNCC, and integer/SIFT initialization. Subset-DIC owns circular sampling, seed selection and reliability propagation. Mesh-DIC owns mesh topology, T3/Q4/Q8 shape functions, nodal initialization, assembly and global solve. Calibration/reconstruction modules consume correspondence fields and remain independent of 2D solver internals.

Supported Workflows

Workflow	Main program	Main output
Mono Subset-DIC	examples/subset_2d.py	sparse 2D displacement and LS strain per deformed frame
Mono Mesh-DIC	examples/mesh_2d.py	T3/Q4/Q8 nodal and dense fields plus nodal LS strain
Stereo 3D-DIC	examples/stereo_3d.py	calibration, 2D fields, 3D points/displacement, face strain
Multi-view 3D-DIC	examples/multiview_3d.py	calibration, masks, pairwise fields/reconstruction, stitched surface

Subset supports first/second-order shape functions, SSD/ZNSSD, ICGN/Forward Gauss-Newton. Mesh supports T3/Q4/Q8, SSD/ZNSSD and global ICGN/Forward GN routes. B-spline degree is 1, 3 or 5. ZNSSD/SSD are lower-is-better; ZNCC is higher-is-better and used for initialization quality.

Installation and Build

Common prerequisites

Install CMake 3.16+, a C++17 compiler, Eigen3, yaml-cpp, Python 3.9+, NumPy, Pillow and PyYAML. OpenCV enables image I/O, SIFT and calibration. Ceres, SciPy and Matplotlib are required only by relevant optimization/reconstruction/plotting paths. pybind11 is required for the Python extension.

Windows with Visual Studio and Conda

- Install Visual Studio Build Tools or Visual Studio with Desktop development with C++ and a Windows SDK.
- Activate the Conda environment containing Python, NumPy, pybind11, Eigen, yaml-cpp and OpenCV.
- Configure from a Visual Studio x64 developer shell:

```
conda activate <environment>
cmd /c 'call "C:\Program Files\Microsoft Visual Studio\18\Community\Common7\Tools\VsDevCmd.bat" -arch=x64 -host_arch=x64'
cmd /c 'call "C:\Program Files\Microsoft Visual Studio\18\Community\Common7\Tools\VsDevCmd.bat" -arch=x64 -host_arch=x64'
```

The post-build command copies `_traditional_dic` into `python/traditional_dic/`. Verify it:

```
python -c "import sys; sys.path.insert(0, 'python'); import traditional_dic; print('OK')"
```

Linux

Install a compiler and development packages, for example `build-essential cmake libeigen3-dev libyaml-cpp-dev python3-dev`. Install Python packages in a virtual environment. Configure and build:

```
python -m venv .venv
source .venv/bin/activate
pip install numpy pillow pyyaml pybind11 scipy matplotlib
cmake -S . -B build -DTRADITIONAL_DIC_BUILD_PYTHON=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
PYTHONPATH=python python -c "import traditional_dic; print('OK')"
```

macOS

Install Xcode Command Line Tools, CMake, Eigen and yaml-cpp with Homebrew, then create a Python virtual environment and install Python dependencies. Use:

```
xcode-select --install
brew install cmake eigen yaml-cpp
python3 -m venv .venv && source .venv/bin/activate
pip install numpy pillow pyyaml pybind11 scipy matplotlib
cmake -S . -B build -DTRADITIONAL_DIC_BUILD_PYTHON=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

On every platform, Python, pybind11 and the compiler architecture must match. If CMake cannot locate a dependency, activate the correct Conda/venv environment or set `CMAKE_PREFIX_PATH`, `Eigen3_DIR`, `yaml-cpp_DIR` and `OpenCV_DIR`.

Configuration and Case Layout

Algorithm files contain parameters only: `subset_2d.yaml`, `mesh_2d.yaml`, `calibration.yaml`, `stereo_3d.yaml`, `multiview_3d.yaml`. All case input/output paths are in `config/case_paths.yaml`.

```
mono_2d: {case_root: case/mono_DIC/ring, images_dir: .}
stereo_3d: {case_root: case/stereo_DIC/plate_center_load, left_images_dir: cam1, right_images_dir: cam2}
multiview_3d: {case_root: case/multi_DIC/CylinderDIC, images: {root: images}, roi: {mode: auto}}
```

Mono: sorted first image is reference, last is ROI, intermediate images are deformed frames. Stereo: first left/right images are reference and last are deformed; calibration directories are set separately. Multi-view auto uses first/last images and generates ROIs; `last_image` uses first=reference, penultimate=deformed, last=ROI.

`result_root` and `visualization_root` default to `result` and `visualization`. Do not place I/O paths in algorithm YAML.

Running Complete Cases

Mono Subset

```
python examples\subset_2d.py --paths-config config\case_paths.yaml --config config\subset_2d.yaml
```

Optional CLI overrides: `--radius`, `--spacing`, `--search-radius`, `--seed-count`, `--max-iterations`. Results: `result/subset/<frame>/displacements.csv`, `stats.json`, `strain.csv`; figures are mirrored under `visualization/subset/<frame>/`.

Mono Mesh

```
python examples\mesh_2d.py --paths-config config\case_paths.yaml --config config\mesh_2d.yaml --element all
```

Choose `--element T3|Q4|Q8|all`. Optional overrides include initialization, optimizer, objective, regularization, dense sample count and quality-control thresholds. Each frame/element writes nodes, elements, `final_U.csv`, `dense_U.csv`, `strain.csv`, summaries and figures.

Stereo 3D

```
python examples\stereo_3d.py --paths-config config\case_paths.yaml --stereo-config config\stereo_3d.yaml --calibration
```

The complete chain calibrates cameras, computes $L0 \rightarrow R0$, $L0 \rightarrow L1$, $L0 \rightarrow R1$, triangulates reference/deformed points and writes 3D displacement. `--skip-calibration` reuses `camera_pair.json`; `--solver mesh` `--element all` runs all mesh elements. Face strain is controlled by `stereo_3d.yaml: strain.enabled`.

Multi-view 3D

```
python examples\multiview_3d.py --paths-config config\case_paths.yaml --config config\multiview_3d.yaml --solver subset
```

The chain self-calibrates, recovers metric scale, selects pairs, prepares ROIs, runs pairwise 2D DIC, reconstructs pair surfaces and stitches them. `--solver` is `subset`, `mesh`, or `both`; `--resume` retains complete 2D pair fields. Pairwise triangular Cosserat strain is controlled by `multiview_3d.yaml: strain.enabled`.

Beginner Checklist

Follow this sequence for a new computer or a new experimental case. Do not skip the import test: a successful CMake build alone does not guarantee that the Python program loads the same extension.

- Clone the repository and create/activate one Python environment.
- Install the Python requirements and C++ dependencies in that environment.
- Configure CMake with `TRADITIONAL_DIC_BUILD_PYTHON=ON`; build the extension.
- Run the import command shown in the build section from the repository root.
- Copy `config/case_paths.yaml` to a new file, for example `config/my_case_paths.yaml`; never modify an algorithm YAML to point at a new case.
- Check image ordering before solving. The programs deliberately infer reference/deformed/ROI roles from the configured directory order.
- Start with one Subset frame using the default configuration and inspect `stats.json`, `displacements.csv`, the field images, and `strain.csv`.

- Only then change search radius, subset radius, shape order, mesh size, strain gauge radius, or quality thresholds.
- Run 3D workflows only after the 2D fields are physically plausible and calibration images/board dimensions are verified.

Minimal Mono Case

Create a directory containing at least three images, such as 001.bmp, 002.bmp, and 003.bmp. With lexicographic ordering, 001.bmp is the reference, 002.bmp is the deformed image, and 003.bmp is the binary ROI. Nonzero ROI pixels are valid. For a temporal sequence, place all deformation images between the reference and ROI, for example 001.bmp, 002.bmp, ..., 100.bmp, 101_roi.bmp.

Set `mono_2d.case_root` and `mono_2d.images_dir`, run the Subset command, then check that every intermediate filename receives its own result directory. If an ROI image is accidentally sorted in the middle, it will be treated as a deformed image; rename it or place images in a separate directory accordingly.

Minimal Stereo Case

The case root needs left and right image directories, one ROI image, and matched calibration-image directories. The first image in each left/right directory is the undeformed stereo pair; the last image is the deformed stereo pair. Calibration directories must contain the same number of corresponding chessboard images. `board.rows`, `board.cols`, and `board.spacing` must describe inner corners and physical spacing in the desired world unit.

Configuration Walkthrough

Subset Parameters

`subset.radius` is the correlation subset radius in pixels. It must contain enough speckle information but remain small relative to displacement gradients. `shape_function.order` selects affine first order or quadratic second order. `optimization.method` selects `icgn` or `forward_gauss_newton`; `correlation.criterion` selects `znssd` or `ssd`. `initialization.integer_search.search_radius` must exceed the expected integer motion. `reliability_propagation.spacing` controls solve-grid spacing only; it is not a strain parameter.

`strain.radius` is independent of subset radius. It is the local virtual strain-gauge radius. Increase it to suppress noise; decrease it to retain strain localization. `min_samples` prevents poorly supported boundary fits. Use `green_lagrange` for finite deformation and `infinitesimal` only when rotations and displacement gradients are small.

Mesh Parameters

`mesh_generation.target_element_size` sets topology density. Smaller elements increase spatial resolution and cost. `optimization.regularization_alpha` smooths displacement gradients but can suppress real local features if too large. `initialization.fedic_fft.window_size` and `search_radius` must be compatible with speckle size and expected motion. Mesh strain uses node connectivity, so there is no strain spacing or radius field; improve strain quality through mesh quality, measurement quality, and regularization.

3D Parameters

`workflow.calibrate`, `compute_fields`, and `reconstruct` decide which stereo stages execute. `reconstruction.max_znssd`, `min_correlation`, and `max_reprojection_error_px` reject unreliable points. Multi-view `camera_pair_selection` chooses adjacent/automatic pairs; `maskGen` controls feature-based ROI support; `scale` must match the physical chessboard. Keep `strain.enabled` true to write face strain for valid triangular reconstructions.

Reproducible Case Procedure

For every reported result, archive the input images, the exact algorithm YAML, the exact path YAML, the Git commit hash, and the generated run summary. Use the following practical sequence:

```
git rev-parse HEAD
python examples\subset_2d.py --paths-config config\my_case_paths.yaml --config config\subset_2d.yaml
python examples\stereo_3d.py --paths-config config\my_case_paths.yaml --stereo-config config\stereo_3d.yaml --calibrat
python examples\multiview_3d.py --paths-config config\my_case_paths.yaml --config config\multiview_3d.yaml --solver su
```

Before accepting a run, check: reference/deformed image roles, ROI overlay, valid-point fraction, displacement direction and magnitude, ZNSSD/reprojection-error distribution, rigid-body test strain, and the expected physical unit after scale recovery. An apparently smooth image is not sufficient evidence of correct calibration or correlation.

Results and Strain

2D CSV fields contain position, displacement, quality and valid. Subset strain uses Ncorr-style local first-order LS planes inside `strain.radius`; `strain.csv` stores fitted gradients, `exx`, `eyy`, `exy`, sample count and validity. Mesh strain uses one-ring node adjacency LS and intentionally has no subset spacing/radius. `measure: green_lagrange` uses $E=0.5(H+H^T+H^TH)$; `infinitesimal` uses the symmetric displacement gradient.

3D strain uses one triangular Cosserat element per reconstructed face: it computes F , $C=F^ATF$, $B=FF^T$, Green-Lagrange and Eulerian-Almansi tensors, principal strains, equivalent strain, shear and area change. The output is `stereo_3d_strain_faces.csv`.

Python API Reference

All arrays are NumPy arrays. Images are 2D grayscale arrays or paths where stated; coordinates are $N \times 2$, 3D points are $N \times 3$, and element indices are zero-based when passed through Python.

Root package

API	Parameters	Returns
<code>subset(reference, deformed, config=None, roi=None, **overrides)</code>	two images; optional YAML dict/path, binary ROI; supported overrides include radius/search/seed/spacing/iterations	dict of <code>x, y, u, v, du_dx, du_dy, dv_dx, dv_dy, correlation, valid</code>
<code>mesh(reference, deformed, nodes, elements, element_type, config=None, **overrides)</code>	images, $N \times 2$ nodes, connectivity, T3/Q4/Q8, config	dict of nodal <code>x, y, u, v, mag, correlation, valid</code>
<code>generate_mesh_from_roi(roi, config=None)</code>	binary ROI and mesh-generation config	nodes/elements mesh data
<code>generate_annulus_meshes_from_mask(mask, config=None)</code>	annular ROI and config	T3/Q4/Q8 mesh collection
<code>reconstruct_from_fields(...), reconstruct_from_field_files(...)</code>	three stereo fields or their CSV directory, cameras, quality options	3D reconstruction result and files

Configuration, core and postprocess

API	Parameters	Returns
<code>load_config(path)</code>	YAML path	mapping

API	Parameters	Returns
<code>normalize_subset_config(config), normalize_mesh_config(config)</code>	mapping or None	backend-ready mapping
<code>core.load_image(path), core.load_mask(path)</code>	file path	backend image/mask
<code>core.normalize_image(image, method='none')</code>	image and normalization name	normalized image
<code>postprocess.save_least_squares_strain_csv(path, points, displacement, radius=None, elements=None, min_samples=6, green_lagrange=True)</code>	Nx2 points/displacement; either subset radius or mesh elements	writes CSV
<code>_traditional_dic.postprocess.compute_least_squares_strain_2d(points, displacement, radius, min_samples=6, green_lagrange=True)</code>	subset field arrays	list of LS strain records
<code>_traditional_dic.postprocess.compute_mesh_least_squares_strain_2d(nodes, displacement, elements, min_samples=3, green_lagrange=True)</code>	mesh arrays	list of LS strain records
<code>_traditional_dic.postprocess.compute_surface_strain(faces, points_ref, points_def, valid_faces)</code>	triangular faces and Nx3 configurations	face strain records

Calibration

`calibration.make_board(config), make_detection_options(config), make_mono_options(config), make_stereo_options(config), make_self_calibration_options(config), and make_scale_options(config)` accept a YAML path or mapping and return backend option objects. `detect_calibration_board(image_path, board=None, config=None, options=None, return_raw=False)` detects board points. `calibrate_mono_zhang(image_paths, board, options=None), calibrate_stereo_zhang(left_paths, right_paths, board, options=None), and calibrate_multiview_colmap_like(image_paths, config=None, options=None, return_raw=False)` return dictionaries. Point-based variants accept image-point/world-point arrays. `estimate_multiview_chessboard_scale(calibration, observations, config=None)` returns scale data. `camera_to_dict, board_to_dict, detection_to_dict, result-to-dict` helpers serialize backend objects; `save_json(data, path)` writes JSON.

Multi-view API

API	Parameters	Returns
<code>select_camera_pairs(calibration, options=None)</code>	calibration mapping/backend and CameraPairSelectionOptions or mapping	CameraPairSelectionResult
<code>generate_pair_masks_from_calibration(case_root, calibration, config=None, pair_selection=None, output_dir=None, options=None)</code>	case, calibration, YAML/options	PairMaskGenerationResult
<code>generate_masks_from_calibration(...)/build_masks_from_calibration(...)</code>	calibration, image shapes/images, mask options	camera masks
<code>compute_pairwise_2d_dic(case_root, calibration=None, config=None, pair_selection=None, subset_config=None, mesh_config=None, output_dir=None, options=None)</code>	case/calibration/config and PairwiseDICOOptions	PairwiseDICRunResult
<code>compute_pairwise_3d_dic(case_root, calibration=None, config=None, pair_selection=None, field_dir=None, output_dir=None, options=None)</code>	pair fields and Pairwise3DOptions	Pairwise3DRunResult
<code>recover_multiview_calibration_scale(case_root, calibration=None, config=None, options=None)</code>	calibration and scale options	MultiviewScaleRecoveryResult
<code>stitch_pairwise_3d_surfaces(case_root, config=None, pair_selection=None, pairwise_3d_dir=None, output_dir=None, options=None)</code>	pair surfaces and stitch options	PairwiseSurfaceStitchRunResult
<code>save_pair_selection_report(result, path)</code>	selection result and JSON path	none
<code>multiview(reference_images, deformed_images=None, calibration=None, solver='subset')</code>	compatibility convenience inputs	workflow result

The option dataclasses are MultiviewMaskOptions, CameraPairSelectionOptions, PairwiseDICOOptions, Pairwise3DOptions, MultiviewScaleRecoveryOptions, and

PairwiseSurfaceStitchOptions. Pass a dataclass or a mapping containing only the fields to override.

Visualization and surface stitching

`visualization_dir_for_result(case_root, result_path, result_root='result')` mirrors an output path. `densify_2d_mesh_displacement_field(nodes, elements, u, v, element_type, valid=None, samples_per_axis=17)` returns a dense field. `plot_2d_field_overlay`, `plot_3d_scatter_field`, `plot_3d_surface_field`, and `plot_stitched_surface_fields` write figures.

`surface_stitching.stitch_surfaces(meshes, min_gap_factor=0.2)` merges `SurfaceMesh` records; `clean_stitched_surface(result, neighbor_count=8, distance_sigma=6, displacement_sigma=6, face_edge_scale=4)` removes outliers; `write_stitch_visualizations(result, output_dir, max_faces=70000)` writes plots.

Troubleshooting

- `ImportError: _traditional_dic`: rebuild with `TRADITIONAL_DIC_BUILD_PYTHON=ON`, then ensure `python/` is on `PYTHONPATH`.
- Missing OpenCV/SIFT: install OpenCV in the active environment and reconfigure CMake.
- Empty/invalid field: inspect ROI ordering, image sequence convention, correlation thresholds, search radius and initialization.
- Noisy strain: increase `Subset strain.radius`; for Mesh improve mesh quality or regularization. Do not use subset spacing as a Mesh strain parameter.
- Multi-view scale failure: verify chessboard images, board dimensions/square size, calibration quality and overlapping camera views.